Module-2
Requirements Engineering

Module-2

- Requirements Engineering
- Establishing the Groundwork
- Eliciting Requirements
- Requirements Analysis
- SRS Documentation
- Metrics in the Process and Project Domains
- Software Measurements
- Software Project Estimation
- Decomposition Techniques
- Empirical Estimation Models
- The Make/Buy Decision.

Requirements Engineering — Elicitation, Analysis & Specification

(The activity of generating the requirements of a system from users, customers, and other stakeholders)

Requirements Elicitation and Analysis

- It's a **process of interacting with customers** and end-users to find out about the domain requirements, what services the system should provide, and the other constraints.

- Domain requirements reflect the environment in which the system operates so, when we talk about an application domain we mean environments such as train operation, medical records, e-commerce etc.

- It may **also involve** different kinds of stockholders; **end-users, managers, system engineers, test engineers, maintenance engineers**, etc.

- A stakeholder is anyone who has direct or indirect influence on the requirements.

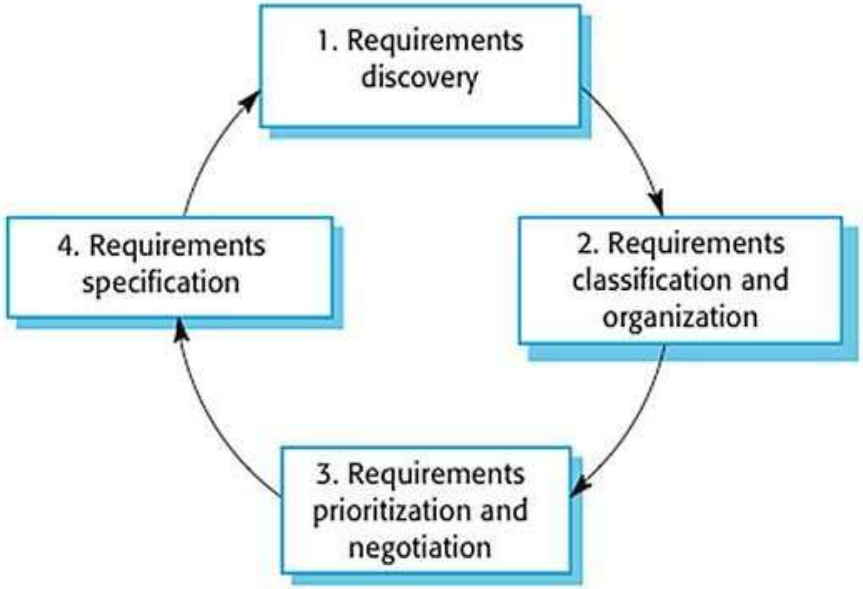
Requirements Elicitation and Analysis

The requirements elicitation and analysis has 4 main process

- We typically start by gathering the requirements, this could be done through a general discussion or interviews with your stakeholders, also it may involve some graphical notation.
- Then you organize the related requirements into sub-components and prioritize them, and finally, you refine them by removing any ambiguous requirements that may arise from some conflicts.

Requirements Elicitation and Analysis

Here are the 4 main processes of requirements elicitation and analysis-



It shows that it's an iterative process with feedback from one activity to another. The process cycle starts with requirements discovery and ends with the requirements document. The cycle ends when the requirements document is complete.

Requirements Elicitation and Analysis

1. Requirements Discovery

- It's the process of interacting with, and gathering the requirements from the stakeholders about the required system and the existing system (if it exists).
- It can be done using some techniques, like interviews, scenarios, prototypes, etc, which help the stockholders to understand what the system will be like.

Gathering and understanding the requirements is a difficult process

- That's because stakeholders may not know what exactly they want the software to do, or they may give unrealistic requirements.
- They may give different requirements, which will result in conflict between the requirements, so we have to discover and resolve these conflicts.

- Also, there might be some factors that influence the stakeholder's decision, like, for For example, managers at a company or professors at the university want to take full control over the management system.

Requirements Elicitation and Analysis

2. Requirements Classification & Organization

- It’s very important to organize the overall structure of the system.
- Putting related requirements together, and decomposing the system into sub-components of related requirements.
- Then, we define the relationship between these components.
- What we do here will help us in the decision of identifying the most suitable architectural design patterns.

Requirements Elicitation and Analysis

3. Requirements Prioritization & Negotiation

- Requirements eliciting and understanding is not an easy process because conflicts may arise due to different stakeholders involved.
- This activity is concerned with prioritizing requirements and finding and resolving requirements conflicts through negotiations until you reach a situation where some of the stakeholders can compromise.
- We shouldn’t reach a situation where a stakeholder is not satisfied because his requirements are not taken into consideration.
- Prioritizing your requirements will help you later to focus on the essentials and core features of the system, so you can meet the user expectations.
- It can be achieved by giving every piece of function a priority level. So, functions with higher priorities need higher attention and focus.

Requirements Elicitation and Analysis

4. Requirements Specification

- It’s the process of writing down the user and system requirements into a document.
- The requirements should be clear, easy to understand, complete, and consistent.
- In practice, this is difficult to achieve as stakeholders interpret the requirements in different ways and there are often inherent conflicts and inconsistencies in the requirements.
- As we’ve mentioned before, the process in requirements engineering is interleaved, and it’s done iteratively. In the first iteration you specify the user requirements, then, specify a more detailed system requirement.

Requirements Elicitation and Analysis

User Requirements

- The *user* requirements for a system should describe the functional and non-functional requirements so that they are understandable by users who don't have technical knowledge.
- You should write user requirements in natural language supplied by simple tables, forms, and intuitive diagrams.
- The requirement document shouldn't include details of the system design, and you shouldn't use any software jargon or formal notations.

Functional vs Non-Functional Requirements

Functional Requirements:

- These are the requirements that the end user specifically demands as basic facilities that the system should offer.
- All these functionalities need to be necessarily incorporated into the system as a part of the contract.
- These are represented or stated in the form of input to be given to the system, the operation performed and the output expected.
- They are basically the requirements stated by the user which one can see directly in the final product, unlike the non-functional requirements.

Functional vs Non-Functional Requirements

Non-Functional Requirements:

- These are basically the quality constraints that the system must satisfy according to the project contract.
- The priority or extent to which these factors are implemented varies from one project to another. They are also called non-behavioral requirements. They basically deal with issues like:

  - Portability
  - Security
  - Maintainability
  - Reliability
  - Scalability
  - Performance
  - Reusability
  - Flexibility

Functional vs Non-Functional Requirements

Functional Requirements	Non-Functional Requirements
A functional requirement defines a system or its components.	A non-functional requirement defines the quality attribute of a software system.
It specifies “What should the software system do?”	It places constraints on “How should the software system fulfil the functional requirements?”
Functional requirement is specified by User.	Non-functional requirement is specified by technical peoples e.g. Architect, Technical leaders and software developers.
It is mandatory.	It is not mandatory.
It is captured in use case.	It is captured as a quality attribute.
Helps you to verify the functionality of the software.	Helps you to verify the performance of the software.

Functional vs Non-Functional Requirements

Functional Requirements	Non-Functional Requirements
Usually easy to define.	Usually more difficult to define.
Example 1) Authentication of the user whenever he/she logs into the system. 2) System shutdown in case of a cyber-attack. 3) A Verification email is sent to the user whenever he/she registers for the first time on some software system.	Example 1) Emails should be sent with a latency of no greater than 12 hours from such an activity. 2) The processing of each request should be done within 10 seconds 3) The site should load in 3 seconds when the The number of simultaneous users is> 10000

Requirement Sources and Elicitation Techniques

System Requirements

- The *system* requirements on the other hand are an expanded version of the user requirements that are used by software engineers as the starting point for the system design.
- They add detail and explain how the user requirements should be provided by the system. They shouldn’t be concerned with how the system should be implemented or designed.
- The system requirements may also be written in natural language but other ways based on structured forms, or graphical notations are usually used.

Module-2
Software Requirement Specifications

Software Requirement Spécifications

- A software requirements specification (SRS) is a document that captures complete description about how the system is expected to perform.
- It is usually signed off at the end of requirements engineering phase.
- This report lays a foundation for software engineering activities and is constructed when entire requirements are elicited and analysed.
- SRS is a formal report, which acts as a representation of software that enables the customers to review whether it (SRS) is according to their requirements. Also, it comprises user requirements for a system as well as detailed specifications of the system requirements.

Software Requirement Spécifications

- The SRS is a specification for a specific software product, program, or set of applications that perform particular functions in a specific environment. It serves several goals depending on who is writing it.
  - First, the SRS could be written by the client of a system.
  - Second, the SRS could be written by a developer of the system.
- The two methods create entirely various situations and establish different purposes for the document altogether.
- The first case, SRS, is used to define the needs and expectations of the users. The second case, SRS, is written for various purposes and serves as a contract document between customer and developer.

What is an SRS?

- SRS is the official statement of what the system developers should implement.
- SRS is a complete description of the behavior of the system to be developed.
- SRS should include both a definition of user requirements and a specification of the system requirements.
- The SRS fully describes what the software will do and how it will be expected to perform.

Purpose of an SRS?

- The SRS precisely defines the software product that will be built.
- SRS used to know all the requirements for the software development and thus that will help in designing the software.
- It provides feedback to the customer. For example :
  - ❖ communication between the Customer, Analyst, system developers, maintainers, ...
  - ❖ contract between Purchaser and Supplier
  - ❖ firm foundation for the design phase
  - ❖ support system testing activities
  - ❖ support project management and control
  - ❖ controlling the evolution of the system

What is not included in an SRS?

Project requirements

- cost, delivery schedules, staffing, reporting procedures

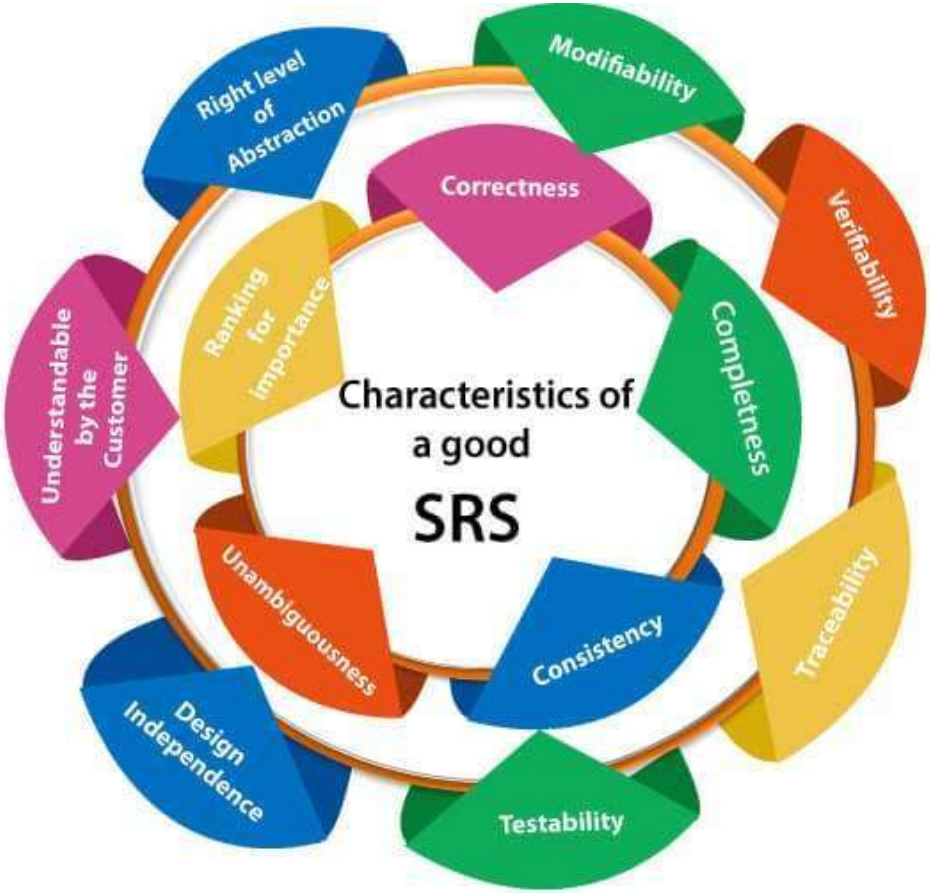
Design solutions

- partitioning of SW into modules, choosing data structures

Product assurance plans

- Quality Assurance procedures, Configuration Management procedures, Verification & Validation procedures

Characteristics of good SRS



Characteristics of good SRS

1. **Correctness:** User review is used to provide the accuracy of requirements stated in the SRS. SRS is said to be perfect if it covers all the needs that are truly expected from the system.

2. **Completeness:** The SRS is complete if, and only if, it includes the following elements:

(1). All essential requirements, whether relating to functionality, performance, design, constraints, attributes, or external interfaces.

(2). Full labels and references to all figures, tables, and diagrams in the SRS and definitions of all terms and units of measure.

3. **Consistency:** The SRS is consistent if, and only if, no subset of individual requirements described in its conflict. There are three types of possible conflict in the SRS:

(1). The specified characteristics of real-world objects may conflicts.

(2). There may be a reasonable or temporal conflict between the two specified actions.

(3). Two or more requirements may define the same real-world object but use different terms for that object.

Characteristics of good SRS

4. Unambiguousness: SRS is unambiguous when every fixed requirement has only one interpretation. This suggests that each element is uniquely interpreted.

5. Ranking for importance and stability: The SRS is ranked for importance and stability if each requirement in it has an identifier to indicate either the significance or stability of that particular requirement.

6. Modifiability: SRS should be made as modifiable as likely and should be capable of quickly obtain changes to the system to some extent. Modifications should be perfectly indexed and cross-referenced.

7. Verifiability: SRS is correct when the specified requirements can be verified with a cost-effective system to check whether the final software meets those requirements. The requirements are verified with the help of reviews.

8. Traceability: The SRS is traceable if the origin of each of the requirements is clear and if it facilitates the referencing of each condition in future development or enhancement documentation.

Essential properties of a good SRS

- **Concise:** The SRS report should be concise and at the same time, unambiguous, consistent, and complete.
- **Structured:** It should be well-structured. A well-structured document is simple to understand and modify.
- **Black-box view:** It should only define what the system should do and refrain from stating how to do these. This means that the SRS document should define the external behavior of the system and not discuss the implementation issues.
- **Conceptual integrity:** It should show conceptual integrity so that the reader can merely understand it. Response to undesired events: It should characterize acceptable responses to unwanted events.
- **Verifiable:** All requirements of the system, as documented in the SRS document, should be correct. This means that it should be possible to decide whether or not requirements have been met in an implementation.

Table of Contents for a SRS Document

Module-2
Software Measurement

Software Measurement:

- A measurement is a manifestation of the size, quantity, amount, or dimension of a particular attribute of a product or process.
- It is an authority within software engineering. The software measurement process is defined and governed by ISO Standard.
- The software measurement process can be characterized by five activities-
    - **Formulation**
    - **Collection**
    - **Analysis**
    - **Interpretation**
    - **Feedback**

Need for Software Measurement:

Software is measured to:

- Create and enhance the quality of the current product or process.
- Anticipate future qualities of the product or process.
- Regulate the state of the project in relation to budget and schedule.
- Identify bottlenecks and areas for improvement to drive process improvement activities.
- Ensure that industry standards and regulations are followed.
- Enable the ongoing improvement of software development practices.

Classification of Software Measurement

There are 2 types of software measurement:

- **Direct Measurement:** In direct measurement, the product, process, or thing is measured directly using a standard scale.

- **Indirect Measurement:** In indirect measurement, the quantity or quality to be measured is measured using related parameters i.e. by use of reference.

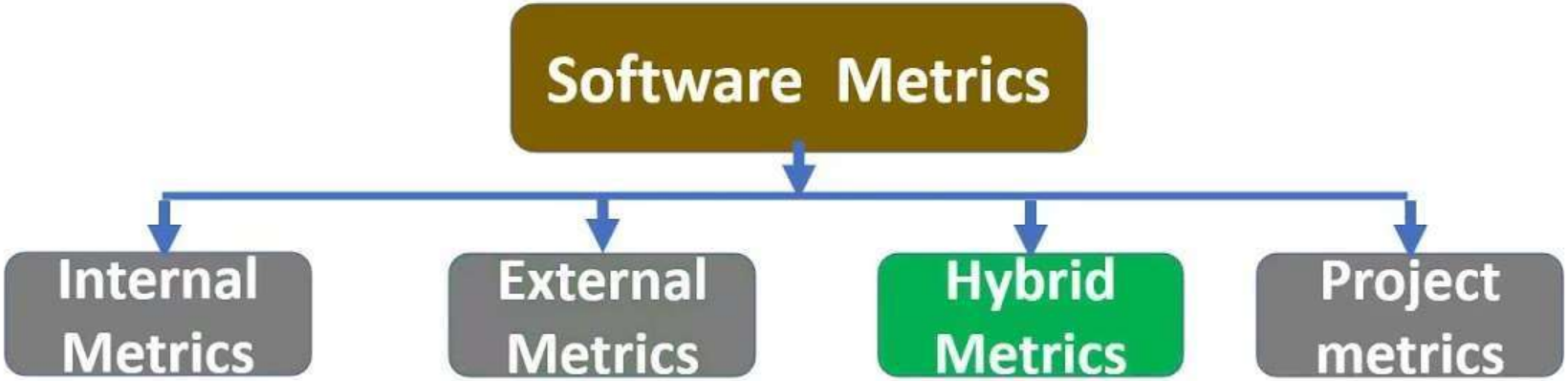
Software Metrics

- A software metric is a measure of software characteristics which are measurable or countable.
- Software metrics are valuable for many reasons, including measuring software performance, planning work items, measuring productivity, and many other uses.
- Within the software development process, many metrics are that are all connected.
- Software metrics are similar to the four functions of management: Planning, Organization, Control, or Improvement.

Characteristics of Software Metrics:

- **Quantitative:** Metrics must possess quantitative nature. It means metrics can be expressed in values.
- **Understandable:** Metric computation should be easily understood, and the method of computing metrics should be clearly defined.
- **Applicability:** Metrics should be applicable in the initial phases of the development of the software.
- **Repeatable:** The metric values should be the same when measured repeatedly and consistent in nature.
- **Economical:** The computation of metrics should be economical.
- **Language Independent:** Metrics should not depend on any programming language.

Types of Software Metrics



- Hybrid metrics: Hybrid metrics are the metrics that combine product, process, and resource metrics.

  For example, cost per Function Point Metric (FP).

Types of Software Metrics

1. **Internal metrics:** Internal metrics are used for measuring properties that are viewed to be of greater importance to a **software developer**. For example, Lines of Code (LOC).

2. **External metrics:** External metrics are used for measuring properties that are viewed to be of greater importance to the **users**, e.g., portability, reliability, functionality, usability, etc.

3. **Hybrid metrics:** Hybrid metrics combine product, process, and resource metrics. For example, cost per FP where FP stands for Function Point Metric.

4. **Project metrics:** Project metrics are used by the **project manager** to check the project's progress.

Classification of Software Metrics

1. Product Metrics
2. Process Metrics
3. Project Metrics

Classification of Software Metrics

1. **Product Metrics:** Product metrics are used to evaluate the state of the product, tracing risks and identify the problem areas. For example-
   - **Size and complexity of software.**
   - **Quality and reliability of software.**
   - **Performance**
   - **Reliability**
   - **Availability**
   - **Usability**
   - **Safety and security**

Classification of Software Metrics

2. Process Metrics: Process metrics pay particular attention to enhancing the long-term process of the team or organization. They are used to measure the characteristics of methods, techniques, and tools that are used for developing software. For example,

- **Efficiency of fault detection**
- **Defects,**
- **Reliability**
- **Maintenance**
- **Management**
- **Effort and schedule variance**

Classification of Software Metrics

3. Project Metrics: They describe the project characteristic and execution process.

Examples-

- **Cycle time**
- **Productivity**
- **Rate of Requirements Change**
- **Estimation Accuracy**
- **Staffing Change Pattern**
- **Cost, Scope, Risk**

Advantages of Software Metrics :

- It helps to identify the particular area for improvising.
- It helps to increase the product quality.
- Managing the workloads and teams.
- Reduction in overall time to produce the product.
- Reduction in cost or budget.
- It helps to determine the complexity of the code and to test the code with resources.
- It helps in providing effective planning, controlling and managing of the entire product.

Disadvantages of Software Metrics :

- It is expensive and difficult to implement the metrics in some cases.
- Performance of the entire team or an individual from the team can't be determined. Only the performance of the product is determined.
- Sometimes the quality of the product is not met with the expectation.
- It leads to measure the unwanted data which is wastage of time.
- Measuring the incorrect data leads to make wrong decision making.

Module-2
Software Project Estimation

Software Project Estimation

- Effective software project estimation is one of the most challenging and important activities in software development. Proper project planning and control is not possible without a sound and reliable estimate.
- Estimation is the process of finding an estimate, or approximation, which is a value that can be used for some purpose even if input data may be incomplete, uncertain, or unstable.
- **Software development estimation revolves around predicting the project's time, effort, cost, and resources; all this information is necessary for the planning stage and to ensure the project's success.**

Software Project Estimation

Estimation is based on −

- **Past Data/Past Experience**
- **Available Documents/Knowledge**
- **Assumptions**
- **Identified Risks**

The four basic steps in Software Project Estimation are −

1. **Estimate the size of the development product.**
2. **Estimate the effort in person-months or person-hours.**
3. **Estimate the schedule in calendar months.**
4. **Estimate the project cost in agreed currency.**

Software Project Estimation

1. **Estimate the size of the development product:**

- Estimation of the size of the software is an essential part of Software Project Management.
- It helps the project manager to further predict the effort and time which will be needed to build the project.
- Various measures are used in project size estimation. Some of these are:
    - **Lines of Code**
    - **Number of entities in ER diagram**
    - **Total number of processes in detailed data flow diagram**
    - **Function points**

Software Project Estimation

- **Lines of Code: Lines of Code (LOC):** As the name suggests, LOC counts the total number of lines of source code in a project.
- **Number of entities in ER diagram:** The number of entities in ER model can be used to measure the estimation of the size of the project. The number of entities depends on the size of the project. This is because more entities needed more classes/structures thus leading to more coding.
- **Total number of processes in detailed data flow diagram:** Data Flow Diagram(DFD) represents the functional view of software. The model depicts the main processes/functions involved in software and the flow of data between them.
- **Function points:** In this method, the number and type of functions supported by the software are utilized to find FPC(function point count).

Software Project Estimation

2. Estimate the effort in person-months or person-hours:

- Once you have an estimate of the size of your product, you can derive the effort estimate.
- This conversion from software size to total project effort can only be done if you have a defined software development lifecycle and development process that you follow to specify, design, develop, and test the software.

There are two main ways to derive effort from size:

- use your organization's own historical data to determine how much effort previous projects of the estimated size have taken.
- use a mature and generally accepted algorithmic approach such as COCOMO model to convert a size estimate into an effort estimate.

Software Project Estimation

3. Estimate the schedule in calendar months:

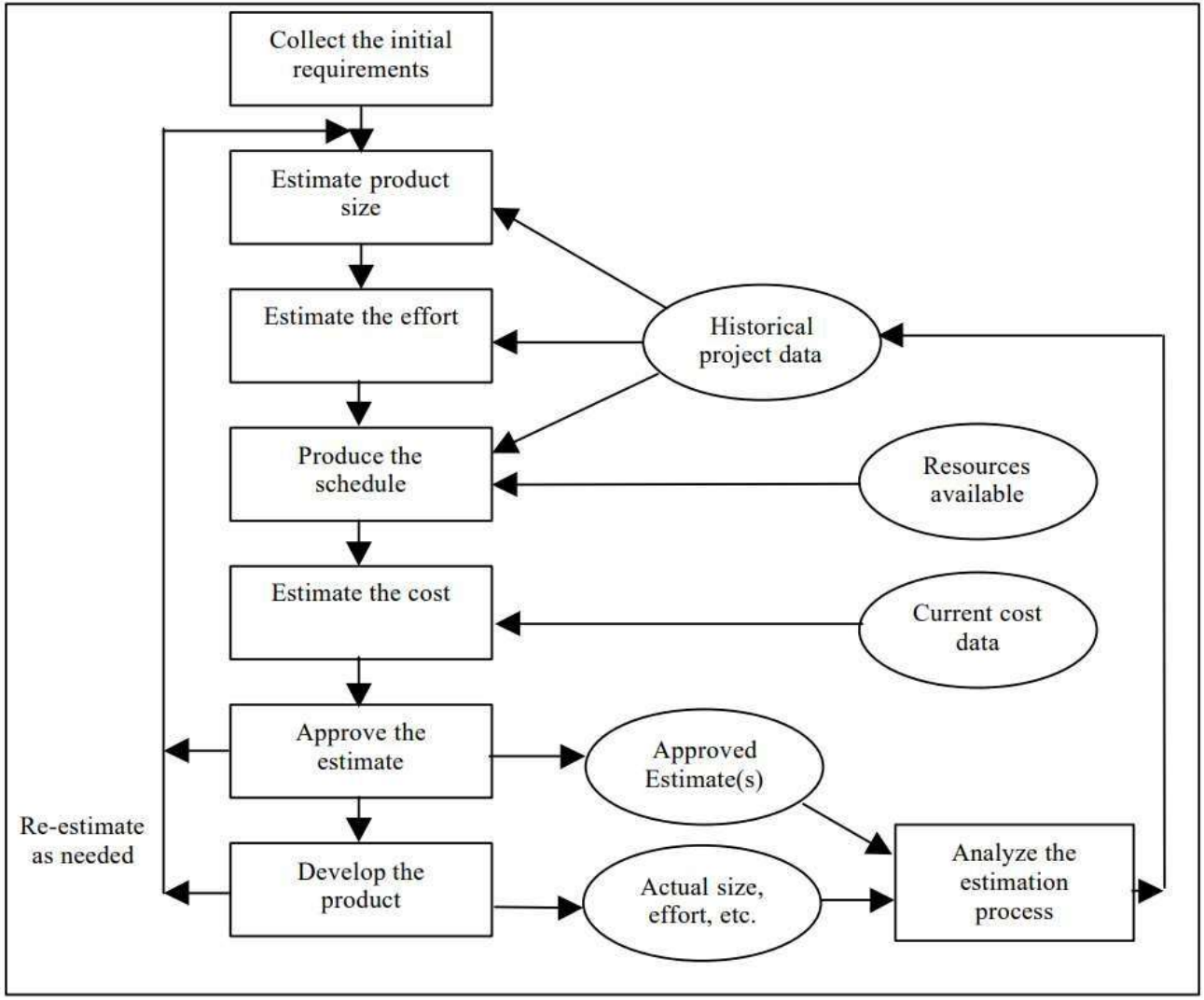
- This generally involves estimating the number of people who will work on the project, what they will work on, when they will start working on the project and when they will finish.
- Again, historical data from your organization's past projects or industry data models can be used to predict the number of people you will need for a project of a given size and how work can be broken down into a schedule.
- If you have nothing else, a schedule estimation rule of thumb can be used to get a rough idea of the total calendar time required:

Schedule in months = 3.0 * (effort-months) 1/3

Software Project Estimation

4. Estimate the project cost in agreed currency:

- There are many factors to consider when estimating the total cost of a project.
- These include labour, hardware and software purchases or rentals, travel for meeting or testing purposes, telecommunications (e.g., longdistance phone calls, video-conferences, dedicated lines for testing, etc.), training courses, office space, and so on.
- You would have to determine what percentage of total project effort should be allocated to each position. Again, historical data or industry data models can help.


Collect the initial requirements
Estimate product size
Estimate the effort
Produce the schedule
Estimate the cost
Approve the estimate
Develop the product
Re-estimate as needed
Historical project data
Resources available
Current cost data
Approved Estimate(s)
Actual size, effort, etc.
Analyze the estimation process

Module-2
Project Estimation Approaches

General Project Estimation Approach: Decomposition Technique.

- The Project Estimation Approach that is widely used is **Decomposition Technique.**
- Decomposition techniques take a divide and conquer approach.
- Size, Effort and Cost estimation are performed in a stepwise manner by breaking down a Project into major Functions or related Software Engineering Activities.

- **Step 1 − Understand the scope of the software to be built.**
- **Step 2 − Generate an estimate of the software size.**
- **Step 3 − Generate an estimate of the effort and cost by breaking down a project into related software engineering activities.**
- **Step 4 − Reconcile estimates: Compare the resulting values of Step 3 and Step 2. If both sets of estimates agree, then your numbers are highly reliable.**

Introduction

- Before an estimate can be made and decomposition techniques applied, the planner must
  - Understand the scope of the software to be built
  - Generate an estimate of the software's size
- Then one of two approaches are used
  - Problem-based estimation
    - Based on either source lines of code or function point estimates
  - Process-based estimation
    - Based on the effort required to accomplish each task

Problem-Based Estimation

1) Start with a bounded statement of scope
2) Decompose the software into problem functions that can each be estimated individually
3) Compute an LOC or FP value for each function
4) Derive cost or effort estimates by applying the LOC or FP values to your baseline productivity metrics (e.g., LOC/person-month or FP/person-month)
5) Combine function estimates to produce an overall estimate for the entire project

Problem-Based Estimation (continued)

- In general, the LOC/pm and FP/pm metrics should be computed by project domain
  - Important factors are team size, application area, and complexity
- LOC and FP estimation differ in the level of detail required for decomposition with each value
  - For LOC, decomposition of functions is essential and should go into considerable detail (the more detail, the more accurate the estimate)
  - For FP, decomposition occurs for the five information domain characteristics and the 14 adjustment factors
    - External inputs, external outputs, external inquiries, internal logical files, external interface files

pm = person month

Process-Based Estimation

1) Identify the set of functions that the software needs to perform as obtained from the project scope
2) Identify the series of framework activities that need to be performed for each function
3) Estimate the effort (in person months) that will be required to accomplish each software process activity for each function

Process-Based Estimation (continued)

4) Apply average labor rates (i.e., cost/unit effort) to the effort estimated for each process activity
5) Compute the total cost and effort for each function and each framework activity (See table in Pressman, p. 655)
6) Compare the resulting values to those obtained by way of the LOC and FP estimates
   - If both sets of estimates agree, then your numbers are highly reliable
   - Otherwise, conduct further investigation and analysis concerning the function and activity breakdown

This is the most commonly used of the two estimation techniques (problem and process)

Reconciling Estimates

- The results gathered from the various estimation techniques must be reconciled to produce a single estimate of effort, project duration, and cost
- If widely divergent estimates occur, investigate the following causes
  - The scope of the project is not adequately understood or has been misinterpreted by the planner
  - Productivity data used for problem-based estimation techniques is inappropriate for the application, obsolete (i.e., outdated for the current organization), or has been misapplied
- The planner must determine the cause of divergence and then reconcile the estimates

Empirical Estimation Models

Introduction

- Estimation models for computer software use empirically derived formulas to predict effort as a function of LOC (line of code) or FP(function point)
- Resultant values computed for LOC or FP are entered into an estimation model
- The empirical data for these models are derived from a limited sample of projects
  - Consequently, the models should be calibrated to reflect local software development conditions

Heuristic Estimation technique: The CoCoMo Model

(The Constructive Cost Model)

COCOMO

- COCOMO is one of the most widely used software estimation models in the world
- It was developed by Barry Boehm in 1981
- COCOMO predicts the effort and schedule for a software product development based on inputs relating to the **size** of the software and a number of **cost drivers** that affect productivity

COCOMO: Some Assumptions

- Primary cost driver is the number of **Delivered Source Instructions** (DSI) or KLOC developed by the project
- COCOMO estimates assume that the project will enjoy **good management** by both the developer and the customer
- Assumes the requirements specification is **not substantially changed** after the plans and requirements phase

Types :

- 1. Organic – A software project is said to be an organic type if the team size required is adequately small, the problem is well understood and has been solved in the past and also the team members have a nominal experience regarding the problem.

- 2. Semi-detached – A software project is said to be a Semi-detached type if the vital characteristics such as team size, experience, and knowledge of the various programming environment lie in between that of organic and Embedded.

- 3. Embedded – A software project requiring the highest level of complexity, creativity, and experience requirement fall under this category. Such software requires a larger team size than the other two models and also the developers need to be sufficiently experienced and creative to develop such complex models.

COCOMO: Three Models

- COCOMO has three different models that reflect the **complexity**:
  - the Basic Model
  - the Intermediate Model
  - and the Detailed Model

Basic COCOMO Model

- Basic COCOMO model estimates the software development effort using only a single predictor variable (size in DSI) and three software development modes

Basic COCOMO Model: Equations

Mode	*Effort*	*Schedule*
Organic	$E=2.4*(KDSI)^{1.05}$	$TDEV=2.5*(E)^{0.38}$
Semidetached	$E=3.0*(KDSI)^{1.12}$	$TDEV=2.5*(E)^{0.35}$
Embedded	$E=3.6*(KDSI)^{1.20}$	$TDEV=2.5*(E)^{0.32}$

2. Intermediate Model –

- 2. Intermediate Model – The basic Cocomo model assumes that the effort is only a function of the number of lines of code and some constants evaluated according to the different software systems. However, in reality, no system's effort and schedule can be solely calculated on the basis of Lines of Code.
- For that, various other factors such as reliability, experience, and Capability. These factors are known as Cost Drivers and the Intermediate Model utilizes 15 such drivers for cost estimation.

3. Detailed Model

- 3. Detailed Model – Detailed COCOMO incorporates all characteristics of the intermediate version with an assessment of the cost driver's impact on each step of the software engineering process.
- The detailed model uses different effort multipliers for each cost driver attribute.
- In detailed cocomo, the whole software is divided into different modules and then we apply COCOMO in different modules to estimate effort and then sum the effort. The Six phases of detailed COCOMO are:
- Planning and requirements
- System design
- Detailed design
- Module code and test
- Integration and test

COCOMO

- Stands for Constructive Cost Model
- Introduced by Barry Boehm in 1981 in his book "Software Engineering Economics"
- Became one of the well-known and widely-used estimation models in the industry
- It has evolved into a more comprehensive estimation model called COCOMO II
- COCOMO II is actually a hierarchy of three estimation models
- As with all estimation models, it requires sizing information and accepts it in three forms: object points, function points, and lines of source code

COCOMO Cost Drivers

- Personnel Factors
  - Applications experience
  - Programming language experience
  - Virtual machine experience
  - Personnel capability
  - Personnel experience
  - Personnel continuity
  - Platform experience
  - Language and tool experience
- Product Factors
  - Required software reliability
  - Database size
  - Software product complexity
  - Required reusability
  - Documentation match to life cycle needs
  - Product reliability and complexity

COCOMO Cost Drivers (continued)

- Platform Factors
  - Execution time constraint
  - Main storage constraint
  - Computer turn-around time
  - Virtual machine volatility
  - Platform volatility
  - Platform difficulty
- Project Factors
  - Use of software tools
  - Use of modern programming practices
  - Required development schedule
  - Classified security application
  - Multi-site development
  - Requirements volatility

COCOMO Model

- **Advantages of the COCOMO model:**

- Provides a systematic way to estimate the cost and effort of a software project.
- Can be used to estimate the cost and effort of a software project at different stages of the development process.
- Helps in identifying the factors that have the greatest impact on the cost and effort of a software project.
- Can be used to evaluate the feasibility of a software project by estimating the cost and effort required to complete it.

COCOMO Model

- **Disadvantages of the COCOMO model:**

- Assumes that the size of the software is the main factor that determines the cost and effort of a software project, which may not always be the case.
- Does not take into account the specific characteristics of the development team, which can have a significant impact on the cost and effort of a software project.
- Does not provide a precise estimate of the cost and effort of a software project, as it is based on assumptions and averages.

Make/Buy Decision

- It is often more cost effective to acquire rather than develop software
- Managers have many acquisition options
  - Software may be purchased (or licensed) off the shelf
  - "Full-experience" or "partial-experience" software components may be acquired and integrated to meet specific needs
  - Software may be custom built by an outside contractor to meet the purchaser's specifications
- The make/buy decision can be made based on the following conditions
  - Will the software product be available sooner than internally developed software?
  - Will the cost of acquisition plus the cost of customization be less than the cost of developing the software internally?
  - Will the cost of outside support (e.g., a maintenance contract) be less than the cost of internal support?

Recommendations

- Do not depend on a single cost or schedule estimate.
- Use several estimating techniques or cost models, compare the results, and determine the reasons for any large variations.
- Document the assumptions made when making the estimates.
- Monitor the project to detect when assumptions that turn out to be wrong jeopardize the accuracy of the estimate.
- Maintain a historical database

Conclusions

- Estimate accuracy is directly proportional to product definition.
  - Before requirements specification, product is very vaguely defined
- More effort, variety of approaches/methods used in estimating = better estimates.
- Use ranges for estimates and gradually refine (tighten) them as the project progresses.
- Measure progress and compare to your historical data
- Refine… Refine… Refine!!!